

Relatório

Alunas: Olívia Izabel Marra Bastos - RA: 22602110, Maria Clara Pereira da Fonseca - RA: 22608328

Título: HelpTech

Descrição:

O HelpTech é um sistema de gerenciamento de uma assistência técnica de dispositivos eletrônicos, desenvolvido em Python utilizando os conceitos de Programação Orientada a Objetos. O sistema permite cadastrar clientes, funcionários e equipamentos, além de criar e acompanhar ordens de serviço.

Por meio de uma ordem de serviço, é possível relacionar um cliente, um equipamento, um funcionário responsável e o serviço que será realizado. O sistema também permite acompanhar o status da ordem de serviço e calcular o valor do serviço.

Atualmente, o sistema trabalha com dois tipos de equipamentos: computadores e celulares.

Justificativa:

Um sistema desse tipo auxilia na categorização, organização e confiabilidade de uma assistência técnica. Facilita o controle dos clientes, funcionários, equipamentos e serviços realizados, reduzindo a necessidade de organizar essas informações manualmente.

Esse sistema organiza qual é cada tipo de equipamento, categoriza níveis de prioridade, problema e garante que cada manutenção seja realizada de maneira rápida e eficiente.

Funcionalidade

O sistema possui as seguintes funcionalidades:

- Cadastro de clientes, funcionários, equipamentos, celulares e computadores;
- Listagem dos clientes cadastrados, funcionários e equipamentos cadastrados;
- Criação de ordens de serviço;
- Registro do serviço realizado;
- Cálculo do valor do serviço;
- Listagem das ordens de serviço;
- Alteração do status da ordem de serviço: Aberta, em andamento, finalizada;

Temas abordados

Função e Método: Função é um bloco de código organizado e reutilizável que executa uma tarefa específica. Quando uma função é definida dentro de uma classe, ela é chamada de método.

Onde foi aplicado: Foram utilizadas várias funções no sistema, principalmente por meio do “def”. Também foram usados métodos especiais como “__init__”, para iniciar os objetos das classes.

Classes: Classes são uma forma de organizar dados e funcionalidades juntos, ao fazer isso é possível organizar o código de forma reutilizável e modular.

Onde foi aplicado: Pessoa, Cliente, Funcionário, Equipamento, Computador, Celular, Equipamento, Item, Ordem_serviço, Serviço.

Encapsulamento: Encapsulamento é esconder e controlar o acesso aos dados internos de um objeto.

Onde foi aplicado: Foi utilizado através de @property e @setter, que controlam o acesso aos atributos internos, como _nome, _cpf, _valor e _marca, permitindo também validar os valores antes de armazená-los.

Herança: Herança é um recurso que permite criar uma classe a partir de outra já existente, a nova classe (classe filha ou subclasse) reutiliza atributos e métodos da classe já existente (classe mãe ou superclasse).

Onde foi aplicado: (Pessoa -> Cliente/ Funcionário) (Equipamento -> Computador/Celular)

Módulo: Módulo é simplesmente um arquivo Python (.py) que contém código reutilizável.

Onde foi aplicado: nos arquivos pessoa.py, main.py, ordem_serviço.py, etc.

Polimorfismo: Polimorfismo são objetos diferentes que podem responder à mesma operação de maneiras diferentes.

Onde foi aplicado: Foi aplicado no método descricao(). Computador e Celular implementam esse método de maneiras diferentes, e o sistema pode chamar equipamento.descricao() sem precisar saber qual é o tipo concreto do equipamento.

Contratos: Um contrato estabelece o que uma classe, função ou método promete fazer.

Onde aplicamos: Foi estabelecido pelos métodos marcados com @abstractmethod, como descricao() em Equipamento, calcular_valor() em Item e exibir_dados() em Pessoa. As subclasses concretas precisam implementar esses métodos.

Protocolos: Um protocolo é um conjunto de operações que um objeto precisa oferecer para ser usado de determinada maneira.

Onde aplicamos: O projeto não utiliza typing.Protocol formalmente. Mas, existe uma ideia de protocolo comportamental, pois o código espera que determinados objetos forneçam métodos como descricao().

Exceções e validações: Uma exceção é um erro ou situação excepcional que ocorre durante a execução do programa.

Onde aplicamos: O projeto utiliza `raise ValueError` para impedir valores inválidos, como nomes vazios, valores negativos e IDs inválidos. Isso é um lançamento de exceções.

Classes abstratas: Classe abstrata é uma classe criada para servir como modelo/base para outras classes.

Onde foi aplicada: Foi aplicada nas classes abstratas Pessoa, Item e Equipamento, que definem características e comportamentos comuns sem fornecer necessariamente sua implementação completa.

Decoradores: Um decorador modifica ou adiciona comportamento a uma função ou classe sem precisar alterar diretamente o corpo dela.

Onde foi aplicado: Usamos o `@property`, `@nome.setter` e `@abstractmethod`. O `@property` é um decorador relacionado a propriedades. O `@nome.setter` é um decorador para definir o setter. O `@abstractmethod` é um decorador para definir métodos abstratos.

Iteradores: É um objeto que permite percorrer uma sequência de elementos, fornecendo um elemento por vez através de `next()`.

Onde foi aplicado: O `"for"` foi usado diversas vezes para percorrer as listas de clientes, funcionários, equipamentos e ordens de serviço. O `"for"` utiliza internamente o mecanismo de interação do Python.

Operadores: Operador é um símbolo ou palavra que realiza uma operação sobre valores ou objetos.

Onde foi aplicado: usamos `"<"`, `"+="`, `"=="`, `"<="` e outros operadores algumas vezes no código. Por exemplo, `"<"` e `"<="` são operadores de comparação, `"+="` é um operador de atribuição composta e `"=="` verifica igualdade.

Tecnologias

- Linguagem Python
- VScode
- GitHub
- Lucidchart

Bibliotecas

abc - Classes abstratas: <https://docs.python.org/pt-br/3/library/abc.html>

enum — Support for enumerations: <https://docs.python.org/3/library/enum.html>

Distribuição de tarefas:

Maria Clara: Código Fonte, Diagrama de Classes

Olivia: Relatório, Repositório Github, Código Fonte

GitHub:

<https://github.com/LivNexus/ProjetoComputacional1>